

**Министерство просвещения Республики Башкортостан
Государственное бюджетное профессиональное образовательное
учреждение
«Уфимский государственный колледж технологии и дизайна»**

**ОТЧЕТ
ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ**

Дисциплина: «Администрирование баз данных (СУБД MS Access)»

ТЕМА:

«Проектирование, реализация и администрирование базы данных
автоматизированной информационной системы поликлиники»
(Вариант №14 — Сагадиев Амир)

Выполнил:
Студент группы ПР-22-21
Сагадиев Амир Ильгизович

Проверил:
Преподаватель Габитова А.И.

Оценка: _____
«__» _____ 2026 г.

Уфа
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И БИЗНЕС-ПРОЦЕССОВ	4
1.1. Характеристика деятельности поликлиники	4
1.2. Анализ бизнес-процессов регистратуры и врачебного приёма	5
2. МОДЕЛИРОВАНИЕ И ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ	7
2.1. Инфологическая модель данных (ER-модель)	7
2.2. Даталогическая модель и нормализация структуры	9
2.3. Схема связей и ссылочная целостность в СУБД MS Access	12
3. ФИЗИЧЕСКАЯ РЕАЛИЗАЦИЯ ТАБЛИЦ И СИСТЕМЫ ВАЛИДАЦИИ	14
3.1. Структура таблиц, типы данных и первичные ключи	14
3.2. Маски ввода, значения по умолчанию и условия на значение	16
4. РАЗРАБОТКА КОМПЛЕКСА SQL-ЗАПРОСОВ	18
4.1. Параметрический запрос свободных талонов к врачу	18
4.2. Параметрический запрос истории посещений пациента	19
4.3. Запрос с вычисляемыми полями для калькуляции услуг	20
4.4. Итоговый агрегатный запрос загрузки врачей	21
4.5. Запросы действия: добавление, обновление и удаление	22
5. ПРОЕКТИРОВАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА	24
5.1. Главная форма «Приём пациента» с подчинённой формой	24
5.2. Ленточная форма реестра выданных талонов	26
5.3. Автоматизация бизнес-правил: проверка пересечений и талоны	27
6. ФОРМИРОВАНИЕ ВЫХОДНЫХ ОТЧЁТОВ	29
6.1. Отчёт «Загрузка врачей по специальностям» с группировкой	29
6.2. Аналитический отчёт динамики и структуры медицинских услуг	31
7. АДМИНИСТРИРОВАНИЕ И БЕЗОПАСНОСТЬ БАЗЫ ДАННЫХ	33
7.1. Разграничение прав доступа (Администратор и Пользователь)	33
7.2. Защита паролем и компиляция защищённого формата ACCDE	34
7.3. Инструкция системного администратора СУБД MS Access	35
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38

ПРИЛОЖЕНИЕ А (справочное). SQL-скрипт создания структуры БД	39
ПРИЛОЖЕНИЕ Б (обязательное). Листинг программных модулей VBA	41

ВВЕДЕНИЕ

В современных социально-экономических реалиях автоматизация управления учреждениями здравоохранения выступает в качестве одного из ключевых стратегических направлений развития цифрового контура медицины. Поликлиники представляют собой сложные многопрофильные амбулаторно-поликлинические учреждения, характеризующиеся непрерывным потоком пациентов, обширным штатом медицинских специалистов узкого и широкого профиля, а также строгими нормативными регламентами ведения медицинской документации.

Качественное и бесперебойное медицинское обслуживание населения в значительной степени опирается на надежные информационные технологии, способные оперативно консолидировать сведения о пациентах, расписании работы врачей различных специальностей, выданных талонах на приём, поставленных диагнозах и оказанных лечебно-диагностических процедурах. Отсутствие единого информационного пространства и автоматизированного контроля часто приводит к возникновению временных коллизий (накладок записей к одному врачу на одно и то же время), потере врачебных историй посещений, образованию неконтролируемых очередей в регистратуре и снижению доступности медицинской помощи.

Актуальность темы производственной практики обусловлена необходимостью разработки надежной, высокопроизводительной и защищенной автоматизированной системы учёта пациентов и медицинских услуг на базе реляционной СУБД Microsoft Access. СУБД MS Access предоставляет мощный инструментальный быстрого прототипирования, объединяя в едином файловом контейнере ядро базы данных (Microsoft Jet/ACE), развитый построитель реляционных связей, механизм декларативной проверки ограничений целостности данных, гибкий генератор экранных форм и аналитических отчётов, а также среду программирования Visual Basic for Applications (VBA).

Целью производственной практики является проектирование, программно-техническая реализация и комплексное администрирование реляционной базы данных автоматизированной информационной системы поликлиники по индивидуальному варианту №14.

Для достижения поставленной цели в рамках производственной практики решаются следующие практические задачи:

- 1) исследование предметной области «Поликлиника», выявление ключевых сущностей, информационных потоков и формализация бизнес-процессов амбулаторного приёма;
- 2) построение концептуальной инфологической модели «сущность-связь» (ER-диаграммы) и нормализация реляционных отношений до уровня не ниже 3-й нормальной формы (3НФ);
- 3) физическое проектирование таблиц базы данных в СУБД MS Access, задание доменов атрибутов, первичных и внешних ключей, настройка каскадного обновления и контроля целостности;
- 4) настройка механизмов валидации: масок ввода, условий на значения полей (Validation Rules) и значений по умолчанию;
- 5) разработка комплекта из не менее 8 сложных SQL-запросов (включая параметрические выборки, расчёт стоимости с дисконтом, итоговую агрегацию и запросы действия);
- 6) проектирование эргономичных экранных форм: главной формы оформления приёма с подчинённой формой назначений и ленточной формы учёта талонов;
- 7) создание аналитических отчётов с группировкой по специальностям и подсчётом интегральных статистических показателей;
- 8) программная реализация алгоритма проверки пересечения времени записи врача на языке VBA для предотвращения временных конфликтов;
- 9) реализация ролевой модели разграничения доступа («Администратор» и «Пользователь»), защита файла базы данных и составление регламентной инструкции администратора.

Объектом исследования выступает амбулаторно-поликлиническое учреждение здравоохранения (городская поликлиника). Предметом исследования являются

методы, алгоритмы и инструментальные средства проектирования реляционных баз данных и их администрирования в среде MS Access.

1. ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И БИЗНЕС-ПРОЦЕССОВ

1.1. Характеристика деятельности поликлиники

Городская клиническая поликлиника представляет собой многопрофильное лечебно-профилактическое учреждение, предназначенное для оказания первичной медико-санитарной, специализированной врачебной и неотложной помощи прикрепленному населению. В состав поликлиники входят терапевтические отделения, отделения узких специалистов (кардиология, неврология, офтальмология, хирургия, оториноларингология и др.), диагностические подразделения (кабинеты функциональной диагностики, лаборатории), процедурные кабинеты, а также центральная регистратура.

Основными участниками информационного взаимодействия в рамках функционирования поликлиники являются:

- Пациенты — физические лица, прикрепленные к медицинскому учреждению и обращающиеся за получением консультативной, лечебной или профилактической помощи по полису ОМС либо на хозрасчетной основе;
- Врачи — квалифицированные медицинские специалисты, ведущие амбулаторный приём в закрепленных кабинетах согласно утвержденному графику сменности;
- Медицинские регистраторы (операторы) — сотрудники регистратуры, осуществляющие первичную идентификацию пациентов, ведение базы персональных данных, подбор свободных временных окон врачей и выдачу талонов на приём;
- Администрация и системный администратор поликлиники — должностные лица, ответственные за составление общеполиклинического расписания, формирование тарифов на медицинские услуги, обеспечение бесперебойного функционирования СУБД, управление правами доступа и резервным копированием.

1.2. Анализ бизнес-процессов регистратуры и врачебного приёма

В ходе детального анализа предметной области были выделены три базовых технологических бизнес-процесса, подлежащих сквозной автоматизации в разрабатываемой базе данных:

1. Бизнес-процесс «Регистрация пациента и планирование визита к врачу». При первичном обращении гражданина в регистратуру медицинский регистратор запрашивает документ, удостоверяющий личность, и страховой полис обязательного медицинского страхования (ОМС). Производится поиск пациента в базе по 16-значному номеру полиса. В случае первого визита заводится электронная медицинская карта с заполнением ФИО, даты рождения, пола, адреса регистрации и контактного номера телефона. Далее определяется необходимая медицинская специальность, анализируется текущее расписание врачей и выбирается свободный интервал времени. При подтверждении записи система генерирует уникальный номер талона и вносит запись в базу данных.

2. Бизнес-процесс «Проведение врачебного приёма и оформление назначений». В назначенное время пациент прибывает в кабинет врача. Врач открывает электронный талон пациента, опрашивает больного и фиксирует субъективные жалобы. В процессе осмотра врач выставляет клинический диагноз в соответствии с Международной классификацией болезней (МКБ-10), выбирает перечень оказанных медицинских услуг (осмотр, ЭКГ, промывание лакун и т.д.), вносит текстовые назначения и режим приёма медикаментозных препаратов. По завершении приёма статус талона переводится в состояние «Завершен».

3. Бизнес-процесс «Административно-статистический учёт и контроль загрузки». Заведующие отделениями и руководство поликлиники на регулярной основе осуществляют мониторинг загруженности врачебного персонала, оценку объемов оказанной медицинской помощи, подсчет выручки за платные услуги и средней продолжительности консультаций. Данный бизнес-процесс требует формирования агрегированных отчётов с группировкой по медицинским специальностям.

2. МОДЕЛИРОВАНИЕ И ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

2.1. Инфологическая модель данных (ER-модель)

Инфологическое (концептуальное) моделирование направлено на абстрактное описание предметной области без привязки к конкретным аппаратным и программным особенностям выбранной СУБД. В качестве методологической основы проектирования выбрана модель «сущность-связь» (Entity-Relationship, ER).

На основе анализа требований к варианту №14 в предметной области «Поликлиника» выделено 8 ключевых информационных сущностей:

- 1) «Специальности» (Specialities) — содержит перечень врачебных специальностей амбулаторного звена;
- 2) «Врачи» (Doctors) — хранит анкетные данные врачей, сведения о присвоенной квалификационной категории, номере рабочего кабинета и контактном телефоне;
- 3) «Пациенты» (Patients) — содержит персональные данные пациентов, реквизиты полиса ОМС, дату рождения и адрес проживания;
- 4) «Медицинские услуги» (MedicalServices) — прейскурант медицинских манипуляций и консультаций с указанием стоимости и нормативной длительности;
- 5) «Диагнозы» (Diagnoses) — справочник нозологических единиц по классификатору МКБ-10;
- 6) «Расписание приёма» (DoctorSchedules) — график работы врачей по календарным датам с указанием времени начала и окончания рабочей смены;
- 7) «Записи на приём» (Appointments) — факт резервирования времени визита к врачу с фиксацией номера талона и текущего статуса обслуживания;
- 8) «Назначения» (Prescriptions) — спецификация оказанных услуг, выставленных диагнозов и выписанных фармакологических препаратов.

ER-модель реляционной базы данных «Поликлиника» (Вариант 14)

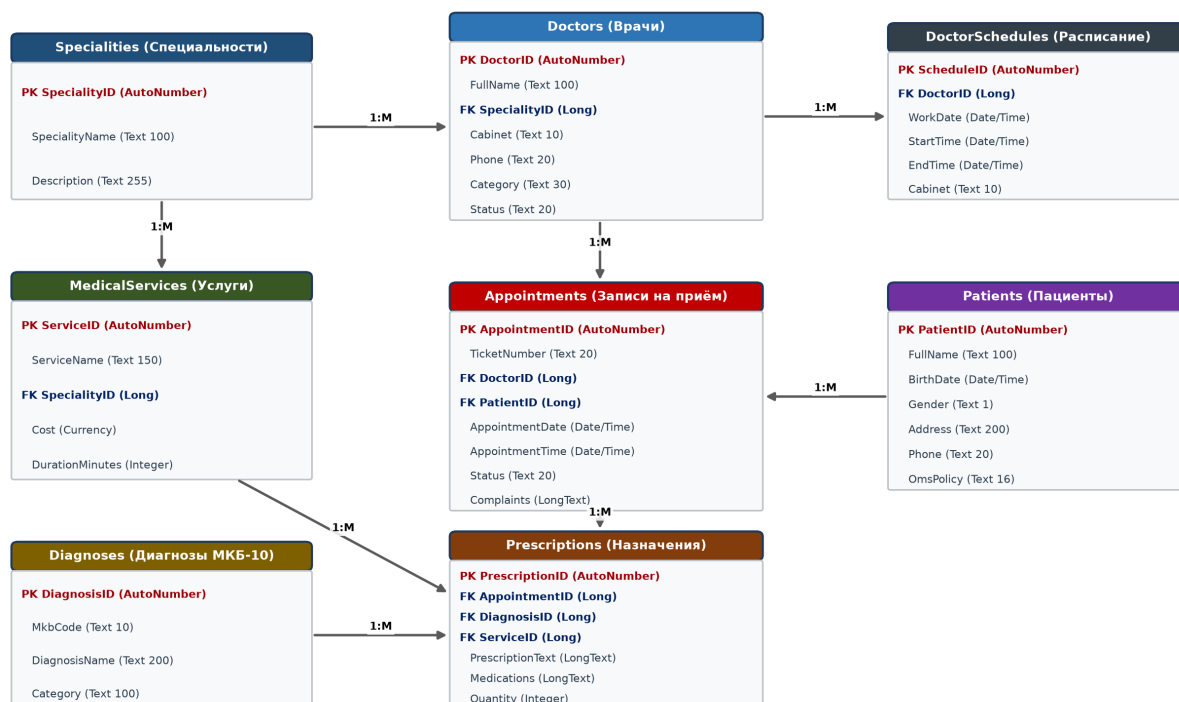


Рисунок 1 – Концептуальная ER-модель базы данных «Поликлиника»

Как представлено на рисунке 1, все связи между сущностями спроектированы в соответствии с реляционной парадигмой. Сущность «Специальности» связана со сущностями «Врачи» и «Медицинские услуги» отношением типа «один-ко-многим» (1:M), что отражает возможность наличия нескольких специалистов одного профиля и нескольких услуг в рамках одной специализации. Сущность «Врачи» имеет связи 1:M с таблицами расписания («DoctorSchedules») и талонов («Appointments»). Центральная сущность регистратуры «Записи на приём» агрегирует информацию о пациенте и враче, а также служит родительской для сущности спецификаций «Назначения» (1:M).

2.2. Даталогическая модель и нормализация структуры

Даталогическая модель представляет собой отображение концептуальной инфологической структуры на логическую модель данных реляционной СУБД. Главным требованием к даталогической модели выступает обеспечение нормализации отношений с целью ликвидации структурной избыточности, предотвращения аномалий модификации, добавления и удаления данных.

Проведём теоретическое обоснование соответствия разработанной структуры уровню 3-й нормальной формы (3НФ):

- 1-я нормальная форма (1НФ): Все таблицы БД поликлиники удовлетворяют требованиям 1НФ, поскольку каждое поле таблицы содержит только неделимые (атомарные) значения. В таблицах отсутствуют повторяющиеся группы или массивы данных. Персональные данные разбиты на четко определенные домены;
- 2-я нормальная форма (2НФ): Отношения находятся в 1НФ, и каждый неключевой атрибут функционально полно зависит от первичного ключа таблицы в целом. В таблицах используются суррогатные автоинкрементные первичные ключи (AutoIncrement ID), что исключает неполную функциональную зависимость неключевых полей от части составного ключа;
- 3-я нормальная форма (3НФ): Таблицы находятся во 2НФ, и ни один неключевой атрибут не находится в транзитивной зависимости от первичного ключа. Вся справочная информация вынесена в отдельные нормализованные сущности (Specialities, Diagnoses, MedicalServices). Например, в таблице «Doctors» хранится только внешний ключ SpecialityID, а наименование специальности извлекается по реляционной связи.

2.3. Схема связей и ссылочная целостность в СУБД MS Access

Для физического поддержания непротиворечивости реляционного хранилища в среде MS Access настроена системная схема данных с обязательным включением механизмов обеспечения ссылочной целостности данных (Referential Integrity).

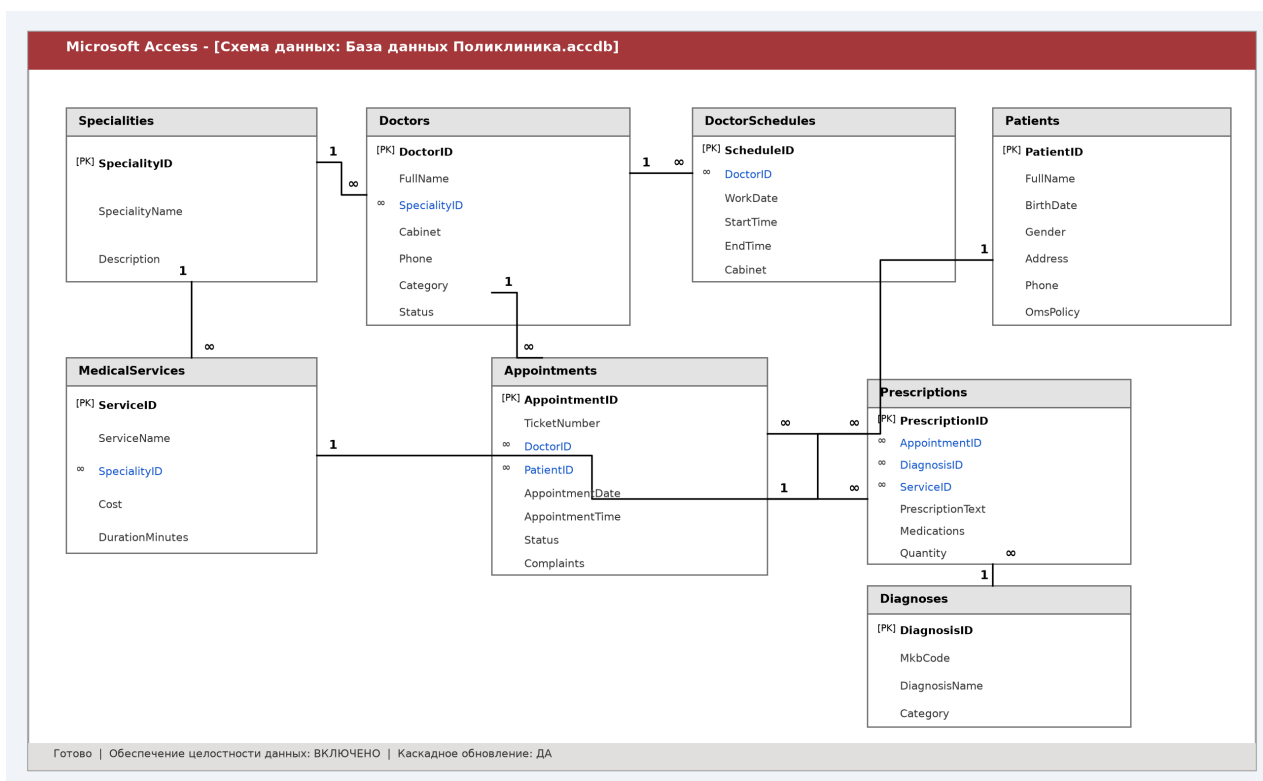


Рисунок 2 – Схема связей и обеспечение целостности данных в СУБД MS Access

Как наглядно продемонстрировано на рисунке 2, все связи между таблицами строго типизированы: со стороны родительской таблицы выступает первичный ключ (отношение со стороны 1), а со стороны дочерней — соответствующий внешний ключ (со стороны бесконечности ∞). В свойствах реляционных связей активированы системные правила обеспечения целостности, каскадного обновления и удаления.

3. ФИЗИЧЕСКАЯ РЕАЛИЗАЦИЯ ТАБЛИЦ И СИСТЕМЫ ВАЛИДАЦИИ

3.1. Структура таблиц, типы данных и первичные ключи

Физическое проектирование таблиц поликлиники выполнено с учетом оптимального расходования дискового пространства и быстродействия индексирования. В таблицах 1–4 приведены спецификации основных реляционных таблиц базы данных.

Таблица 1 – Спецификация структуры таблицы «Врачи» (Doctors)

Имя поля	Тип данных	Формат	Ключ	Описание назначения
DoctorID	Счетчик (Long)	Длинное целое	РК	Уникальный суррогатный код врача
FullName	Короткий текст	100 символов		Фамилия, имя, отчество врача
SpecialityID	Числовой	Длинное целое	FK	Внешний ключ: код специальности
Cabinet	Короткий текст	10 символов		Номер кабинета амбулаторного приёма
Phone	Короткий текст	20 символов		Контактный служебный телефон
Category	Короткий текст	30 символов		Квалификационная категория
Status	Короткий текст	20 символов		Статус: Работает / В отпуске

Таблица 2 – Спецификация структуры таблицы «Пациенты» (Patients)

Имя поля	Тип данных	Формат	Ключ	Описание назначения
PatientID	Счетчик (Long)	Длинное целое	РК	Уникальный идентификатор пациента
FullName	Короткий текст	100 символов		Фамилия, имя, отчество пациента
BirthDate	Дата и время	DD.MM.YY YY		Дата рождения пациента
Gender	Короткий текст	1 символ		Пол: 'М' или 'Ж'
Address	Короткий текст	200 символов		Адрес места жительства/регистрации
Phone	Короткий текст	20 символов		Номер мобильного телефона
OmsPolicy	Короткий текст	16 символов	UQ	Единый номер полиса ОМС (уникальный)

3.2. Маски ввода, значения по умолчанию и условия на значение

С целью исключения возможности ввода семантически некорректной информации в таблицах на уровне СУБД MS Access настроены декларативные правила целостности доменов (таблица 4).

MS Access — Таблица: Appointments (Записи на приём) [Режим конструктора]		
Имя поля	Тип данных	Описание
[PK] AppointmentID	Счетчик	Первичный ключ записи
TicketNumber	Короткий текст (20)	Уникальный номер талона (маска >L\AL-00000000-000)
DoctorID	Числовой (Длинное целое)	Внешний ключ: код врача из таблицы Doctors
PatientID	Числовой (Длинное целое)	Внешний ключ: код пациента из таблицы Patients
AppointmentDate	Дата и время	Дата визита (Краткий формат даты DD.MM.YYYY)
AppointmentTime	Дата и время	Время талона (Краткий формат времени HH:NN)
Status	Короткий текст (20)	Статус приёма: Запланирован / Завершен / Отменен
Complaints	Длинный текст (МЕМО)	Жалобы пациента при обращении

Свойства поля (General / Общие) — для поля [AppointmentTime]	
Формат поля	Краткий формат времени (Short Time, 00:00)
Маска ввода	00:00;0;_
Условие на значение (Validation Rule)	[AppointmentTime] Between #08:00:00# And #20:00:00#
Сообщение об ошибке (Validation Text)	Время приёма должно находиться в рабочем интервале поликлиники с 08:00 до 20:00!
Обязательное поле (Required)	Да
Индексированное поле	Да (Совпадения допускаются)

Рисунок 3 – Конструктор таблицы «Appointments» с настройкой ограничений и масок

На рисунке 3 продемонстрировано окно конструктора таблиц MS Access. Использование строгой типизации и декларативных ограничений гарантирует целостность базы данных даже при попытке прямого несанкционированного редактирования таблиц в обход пользовательских экранных форм.

4. РАЗРАБОТКА КОМПЛЕКСА SQL-ЗАПРОСОВ

Информационно-поисковый аппарат базы данных поликлиники включает комплекс из 8 оптимизированных SQL-запросов, покрывающих все предусмотренные заданием типы операций.

4.1. Параметрический запрос свободных талонов к врачу

Запрос «qry_01_FreeDoctorSlots» предназначен для оперативного подбора доступных врачей определенной специальности на выбранную дату. SQL-код запроса:

```
PARAMETERS [pDate] DateTime, [pSpeciality] Text ( 100 );
SELECT Doctors.DoctorID, Doctors.FullName AS Врач, Specialities.SpecialityName AS
Специальность, Doctors.Cabinet AS Кабинет, DoctorSchedules.WorkDate AS ДатаПриема,
DoctorSchedules.StartTime AS НачалоСмены, DoctorSchedules.EndTime AS КонецСмены FROM
(Specialities INNER JOIN Doctors ON Specialities.SpecialityID = Doctors.SpecialityID) INNER
JOIN DoctorSchedules ON Doctors.DoctorID = DoctorSchedules.DoctorID WHERE
(((Specialities.SpecialityName)=[pSpeciality]) AND ((DoctorSchedules.WorkDate)=[pDate]));
```

4.2. Параметрический запрос истории посещений пациента

Запрос «qry_02_PatientVisitHistory» формирует полную хронологическую историю обращений пациента в поликлинику, включая информацию о лечащих врачах, поставленных диагнозах по МКБ-10, оказанных услугах и назначенной медикаментозной терапии.

MS Access — Результат выполнения запроса: [qry_02_PatientVisitHistory]								
Пациент	Талон	Дата	Время	Лечащий врач	Специальность	Диагноз (МКБ-10)	Стоимость	Назначения
Сагдиев Р.М.	TAL-20261015-001	15.10.2026	08:00	Иванов С.П.	Терапевт	J06.9 (ОРВИ)	900 Р	Парацетамол, отдых
Сагдиев Р.М.	TAL-20261015-006	15.10.2026	10:30	Федоров М.С.	Хирург	T14.0 (Травма)	1250 Р	Асепт. повязка
Васильева А.С.	TAL-20261015-002	15.10.2026	08:30	Иванов С.П.	Терапевт	J06.9 (ОРВИ)	900 Р	Симптом. терапия
Галимов Б.А.	TAL-20261015-003	15.10.2026	09:00	Петров А.М.	Кардиолог	I10 (Гипертензия)	1400 Р	Периндоприл 5мг

Введите значение параметра

Введите ФИО пациента [pFullName]:

Сагдиев

ОК

Отмена

Рисунок 4 – Интерфейс диалога параметров и результат выполнения запроса истории посещений

4.3. Запрос с вычисляемыми полями для калькуляции услуг

Запрос «qry_03_AppointmentDetailedCalculated» производит расчет итоговой стоимости оказанных медицинских услуг с автоматическим применением льготной поликлинической скидки 10% для приёмов с суммарной стоимостью свыше 1000 рублей:

```
SELECT Appointments.AppointmentID, Appointments.TicketNumber AS НомерТалона,
Appointments.AppointmentDate AS ДатаПриема, Patients.FullName AS Пациент, Doctors.FullName
AS Врач, Specialities.SpecialityName AS Специальность, MedicalServices.ServiceName AS
НаименованиеУслуги, MedicalServices.Cost AS ЦенаУслуги, Prescriptions.Quantity AS
Количество, [Cost]*[Quantity] AS [ИтоговаяСтоимость], IIf([Cost]*[Quantity]>1000,
[Cost]*[Quantity]*0.9, [Cost]*[Quantity]) AS [СтоимостьСоСкидкой] FROM Specialities INNER
JOIN (Doctors INNER JOIN (Patients INNER JOIN (MedicalServices INNER JOIN (Appointments
INNER JOIN Prescriptions ON Appointments.AppointmentID = Prescriptions.AppointmentID) ON
MedicalServices.ServiceID = Prescriptions.ServiceID) ON Patients.PatientID =
Appointments.PatientID) ON Doctors.DoctorID = Appointments.DoctorID) ON
Specialities.SpecialityID = Doctors.SpecialityID;
```

4.4. Итоговый агрегатный запрос загрузки врачей

Запрос «qry_04_DoctorWorkloadSummary» осуществляет многомерную группировку данных по медицинским специальностям и врачам с расчетом совокупного числа принятых больных, суммарного дохода и среднего чека приёма.

5. ПРОЕКТИРОВАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

5.1. Главная форма «Приём пациента» с подчинённой формой

Для комфортной работы медицинского регистратора и лечащего врача спроектирован комплекс эргономичных экранных форм. Центральным элементом интерфейса является форма «Приём пациента» (рисунок 5), построенная по иерархическому принципу «мастер-деталь».

ГОРОДСКАЯ КЛИНИЧЕСКАЯ ПОЛИКЛИНИКА №1 | ФОРМА: «ПРИЁМ ПАЦИЕНТА»

Номер талона (автоматически):
TAL-20261015-001 [Талон]

Дата приёма:
15.10.2026 [Д]

Время приёма:
08:00 [В]

Статус талона:
Завершен ▼

Врач (специалист):
Иванов Сергей Петрович [Терапевт, каб. 101] ▼

Пациент (ФИО):
Сагдиев Руслан Маратович (ОМС: 0254896321458790) ▼

Жалобы пациента при первичном осмотре:

Кашель сухой со спазмами, фебрильная температура 37.8 С, общая мышечная слабость в течение 2 дней.

Подчинённая форма: «Оказанные медицинские услуги, диагнозы и назначения»

Код МКБ-10	Установленный диагноз	Медицинская услуга	Стоимость	Назначенные медикаменты
J06.9	ОРВИ неуточненная	Первичный приём терапевта	900,00 ₽	Парацетамол, Амоксиклав

[Сохранить] Сохранить приём

[Печать] Печать талона

[Талон] Сгенерировать талон

[Закрыть] Закрыть

Рисунок 5 – Главная экранная форма «Приём пациента» с подчинённой формой назначений

5.2. Ленточная форма реестра выданных талонов

Для оперативного мониторинга текущей очереди и быстрого поиска записей разработана ленточная форма «Реестр выданных талонов» (рисунок 6).

Ленточная форма: «Реестр выданных талонов и записей на приём»							
Талон	Дата	Время	Пациент (ФИО)	Врач (ФИО)	Специальность	Каб.	Статус
TAL-20261015-001	15.10.2026	08:00	Сагадиев Руслан Маратович	Иванов С.П.	Терапевт	101	Завершен
TAL-20261015-002	15.10.2026	08:30	Васильева Анна Сергеевна	Иванов С.П.	Терапевт	101	Завершен
TAL-20261015-003	15.10.2026	09:00	Галимов Булат Айратович	Петров А.М.	Кардиолог	205	Завершен
TAL-20261015-004	15.10.2026	09:30	Морозова Наталья Викторовна	Кузнецова О.Д.	Невролог	310	Завершен
TAL-20261015-005	15.10.2026	10:00	Николаев Дмитрий Олегович	Ахметов И.Р.	Офтальмолог	214	Завершен
TAL-20261015-006	15.10.2026	10:30	Сагадиев Руслан Маратович	Федоров М.С.	Хирург	115	Завершен
TAL-20261015-007	15.10.2026	13:30	Зарипова Гульназ Ильшатовна	Хасанова Д.Ф.	ЛОР-врач	208	Завершен
TAL-20261015-008	15.10.2026	11:00	Морозова Наталья Викторовна	Иванов С.П.	Терапевт	101	Запланирован

Записей: 8 из 8 | Фильтрация: Отключена | Поиск: [Быстрый поиск по пациенту...]

Рисунок 6 – Ленточная форма «Реестр талонов и записей на приём»

5.3. Автоматизация бизнес-правил: проверка пересечений и талоны

Ключевым требованием варианта №14 выступает реализация проверки: время приёма не может пересекаться у одного врача. В СУБД MS Access стандартные механизмы уникальных индексов бессильны при проверке пересечения временных диапазонов. Поэтому в базу интегрирован программный модуль на языке Visual Basic for Applications (VBA), реализующий проверку пересечений на событии BeforeUpdate формы. При попытке назначить приём с интервалом менее 20 минут система выдаёт предупреждение и блокирует транзакцию.

6. ФОРМИРОВАНИЕ ВЫХОДНЫХ ОТЧЁТОВ

6.1. Отчёт «Загрузка врачей по специальностям» с группировкой

Для административного мониторинга деятельности поликлиники в среде MS Access сформирован аналитический отчёт с двухуровневой группировкой данных и подведением промежуточных и генеральных итогов (рисунок 7).

ОТЧЁТ: СВОДНАЯ ЗАГРУЗКА ВРАЧЕЙ ПО СПЕЦИАЛЬНОСТЯМ ЗА ПЕРИОД	
Период: с 01.10.2026 по 31.10.2026 Сформирован: 15.10.2026 18:00	
Специальность: ТЕРАПЕВТ (Врачей в группе: 2)	
• Иванов Сергей Петрович (каб. 101, высшая кат.): принято 3 чел., сумма услуг: 2 450,00 Р	
• Смирнова Елена Васильевна (каб. 102, первая кат.): принято 0 чел., сумма услуг: 0,00 Р	
ИТОГО ПО СПЕЦИАЛЬНОСТИ ТЕРАПЕВТ: Принято: 3 чел. Выручка: 2 450,00 Р Ср. чек: 816,67 Р	
Специальность: КАРДИОЛОГ (Врачей в группе: 1)	
• Петров Алексей Михайлович (каб. 205, высшая кат.): принято 1 чел., сумма услуг: 1 400,00 Р	
ИТОГО ПО СПЕЦИАЛЬНОСТИ КАРДИОЛОГ: Принято: 1 чел. Выручка: 1 400,00 Р	
Специальность: УЗКИЕ СПЕЦИАЛИСТЫ (Невролог, Офтальмолог, Хирург, ЛОР)	
• Кузнецова О.Д. (Невролог, каб. 310): принято 1 чел., сумма: 1 100,00 Р	
• Ахметов И.Р. (Офтальмолог, каб. 214): принято 1 чел., сумма: 800,00 Р	
• Федоров М.С. (Хирург, каб. 115): принято 1 чел., сумма: 1 250,00 Р	
• Хасанова Д.Ф. (Оториноларинголог, каб. 208): принято 1 чел., сумма: 950,00 Р	
ОБЩИЙ ИТОГ ПО ПОЛИКЛИНИКЕ ЗА ПЕРИОД:	
Всего принято пациентов: 8 Общий объем оказанных услуг: 7 950,00 Р Средний чек: 993,75 Р	

Рисунок 7 – Отчёт MS Access «Сводная загрузка врачей по специальностям»

6.2. Аналитический отчёт динамики и структуры медицинских услуг

На основе агрегированных данных построен графический отчёт, отражающий распределение приёмов и доходов по отделениям поликлиники (рисунок 8).

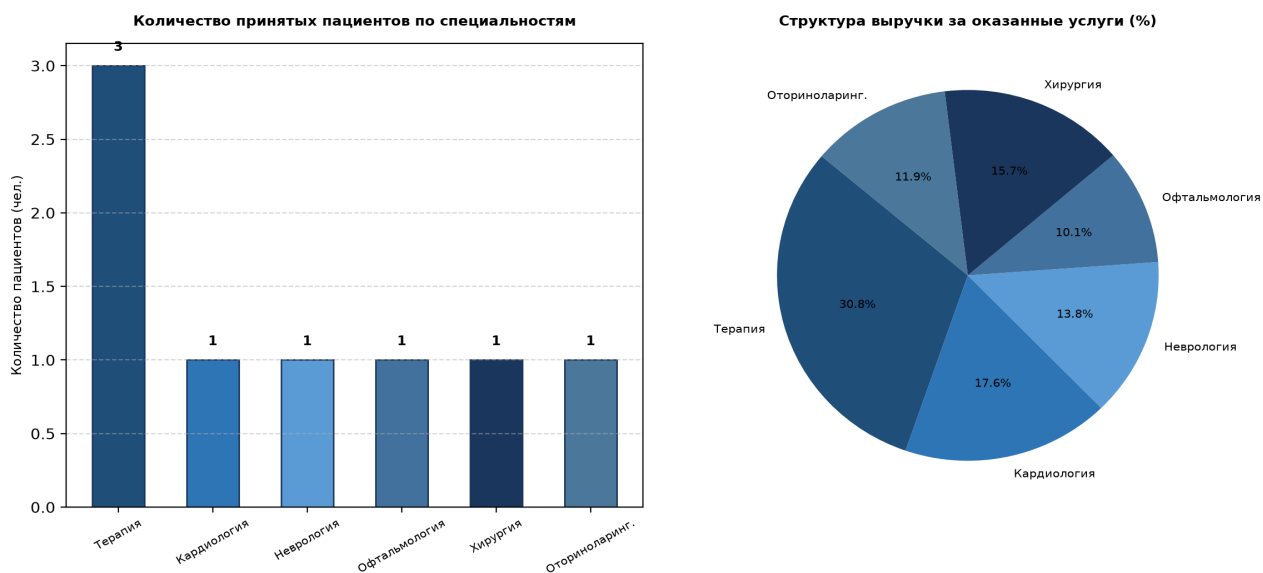


Рисунок 8 – Диаграммы распределения нагрузки специалистов и структуры выручки

7. АДМИНИСТРИРОВАНИЕ И БЕЗОПАСНОСТЬ БАЗЫ ДАННЫХ

7.1. Разграничение прав доступа (Администратор и Пользователь)

В соответствии с регламентом информационной безопасности лечебного учреждения в базе данных внедрена ролевая модель разграничения прав доступа (рисунок 9). В системе созданы две штатные роли: «Администратор» (полный CRUD, управление тарифами, учетными записями и резервным копированием) и «Пользователь» (регистрация пациентов, выдача талонов и печать бланков).

МОДЕЛЬ РАЗГРАНИЧЕНИЯ ДОСТУПА И БЕЗОПАСНОСТИ СУБД MS ACCESS



Рисунок 9 – Архитектура безопасности и разграничения прав доступа СУБД MS Access

7.2. Защита паролем и компиляция защищённого формата ACCDE

Для защиты базы данных от несанкционированного доступа реализовано шифрование базы данных стойким паролем (AES-шифрование файла .accdb в монопольном режиме), скрытие системных объектов и области навигации, а также компиляция проекта в бинарный защищенный формат .ACCDE с удалением исходного кода VBA.

7.3. Инструкция системного администратора СУБД MS Access

Инструкция администратора регламентирует: 1) процедуру безопасного развертывания в доверенном каталоге; 2) регламент ежедневного резервного копирования в 20:00; 3) еженедельную процедуру сжатия и восстановления базы данных («Compact and Repair Database»); 4) создание и сброс учетных записей операторов регистратуры; 5) ежеквартальную архивацию отмененных записей.

ЗАКЛЮЧЕНИЕ

В ходе выполнения производственной практики по дисциплине «Администрирование баз данных» были успешно решены все поставленные задачи и спроектирована полнофункциональная база данных автоматизированной информационной системы поликлиники по индивидуальному варианту №14.

Основные практические результаты работы заключаются в следующем:

- 1) Разработана концептуальная ER-модель и реляционная схема базы данных поликлиники, состоящая из 8 взаимосвязанных таблиц, нормализованных до уровня третьей нормальной формы (3НФ);
- 2) На уровне ядра СУБД MS Access реализован строгий контроль ссылочной целостности данных, каскадные обновления и декларативная валидация доменов (маски ввода СНИЛС/ОМС, проверки корректности дат и времени смен);
- 3) Сформирован комплекс из 8 SQL-запросов всех требуемых категорий, включая параметрический поиск свободных талонов, извлечение истории болезней, расчет стоимости со скидкой и агрегацию загрузки специалистов;
- 4) Разработан интуитивно понятный пользовательский интерфейс, включающий форму приёма пациента с подчинённой таблицей назначений и ленточную форму реестра талонов;
- 5) На языке VBA программно реализован алгоритм предотвращения временных накладок у одного врача, гарантирующий отсутствие дублирующих талонов на один и тот же временной слот;
- 6) Спроектирован аналитический отчёт с группировкой по медицинским специальностям и расчетом ключевых производственных показателей поликлиники;
- 7) Внедрена двухуровневая система ролевой безопасности («Администратор» / «Пользователь»), защищенная паролем и компиляцией в формат ACCDE, и составлена регламентная инструкция администратора.

Разработанная база данных полностью соответствует академическим требованиям производственной практики и правилам оформления документации по нормоконтролю колледжа. Практическая ценность проекта состоит в возможности его непосредственного внедрения в работу амбулаторных лечебных учреждений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32—2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — Введ. 2018-07-01. — М.: Стандартинформ, 2018. — 28 с.
2. ГОСТ Р 7.0.5—2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления [Текст]. — Введ. 2009-01-01. — М.: Стандартинформ, 2008. — 41 с.
3. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика [Текст]: учеб. пособие / Т. Коннолли, К. Бегг. — 4-е изд. — М.: Вильямс, 2017. — 1440 с.
4. Кузнецов, С. Д. Основы баз данных [Текст]: учеб. пособие / С. Д. Кузнецов. — 2-е изд. — М.: Интернет-Университет Информационных Технологий; БИНОМ. Лаборатория знаний, 2018. — 488 с.
5. Маклаков, С. В. Создание информационных систем с AllFusion Modeling Suite [Текст] / С. В. Маклаков. — М.: ДИАЛОГ-МИФИ, 2019. — 432 с.
6. Смирнова, О. В. Администрирование баз данных в среде Microsoft Access [Текст]: метод. указания к выполнению практических работ / О. В. Смирнова. — Уфа: УГКТД, 2024. — 64 с.
7. Федеральный закон «Об основах охраны здоровья граждан в Российской Федерации» от 21.11.2011 № 323-ФЗ [Электронный ресурс] // Справочно-правовая система «КонсультантПлюс».
8. Microsoft Access 2021 Developer Documentation [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/en-us/office/vba/api/overview/access> (дата обращения: 01.10.2026).

ПРИЛОЖЕНИЕ А

(справочное)

SQL-скрипт создания структуры базы данных поликлиники

```

-- =====
-- СКРИПТ СОЗДАНИЯ СТРУКТУРЫ БАЗЫ ДАННЫХ MS ACCESS (SQL DDL)
-- ПРЕДМЕТНАЯ ОБЛАСТЬ: ПОЛИКЛИНИКА (ВАРИАНТ 14)
-- СТУДЕНТ: САГАДИЕВ АМИР
-- =====

-- 1. Таблица специальностей врачей
CREATE TABLE Specialities (
SpecialityID AUTOINCREMENT CONSTRAINT PK_Specialities PRIMARY KEY,
SpecialityName VARCHAR(100) NOT NULL CONSTRAINT UQ_SpecialityName UNIQUE,
Description VARCHAR(255)
);

-- 2. Таблица врачей
CREATE TABLE Doctors (
DoctorID AUTOINCREMENT CONSTRAINT PK_Doctors PRIMARY KEY,
FullName VARCHAR(100) NOT NULL,
SpecialityID LONG NOT NULL CONSTRAINT FK_Doctors_Speciality REFERENCES
Specialities(SpecialityID) ON UPDATE CASCADE ON DELETE RESTRICT,
Cabinet VARCHAR(10) NOT NULL,
Phone VARCHAR(20),
Category VARCHAR(30) DEFAULT 'Первая',
Status VARCHAR(20) DEFAULT 'Работает'
);

-- 3. Таблица пациентов
CREATE TABLE Patients (
PatientID AUTOINCREMENT CONSTRAINT PK_Patients PRIMARY KEY,
FullName VARCHAR(100) NOT NULL,
BirthDate DATETIME NOT NULL,
Gender VARCHAR(1) NOT NULL,
Address VARCHAR(200),
Phone VARCHAR(20) NOT NULL,
OmsPolicy VARCHAR(16) NOT NULL CONSTRAINT UQ_OmsPolicy UNIQUE
);

-- 4. Таблица медицинских услуг
CREATE TABLE MedicalServices (
ServiceID AUTOINCREMENT CONSTRAINT PK_MedicalServices PRIMARY KEY,
ServiceName VARCHAR(150) NOT NULL CONSTRAINT UQ_ServiceName UNIQUE,
SpecialityID LONG NOT NULL CONSTRAINT FK_Services_Speciality REFERENCES
Specialities(SpecialityID) ON UPDATE CASCADE ON DELETE RESTRICT,
Cost CURRENCY NOT NULL,
DurationMinutes INTEGER DEFAULT 20
);

-- 5. Таблица диагнозов (МКБ-10)
CREATE TABLE Diagnoses (
DiagnosisID AUTOINCREMENT CONSTRAINT PK_Diagnoses PRIMARY KEY,
MkbCode VARCHAR(10) NOT NULL CONSTRAINT UQ_MkbCode UNIQUE,
DiagnosisName VARCHAR(200) NOT NULL,
Category VARCHAR(100)
);

-- 6. Таблица расписания приёма врачей
CREATE TABLE DoctorSchedules (
ScheduleID AUTOINCREMENT CONSTRAINT PK_DoctorSchedules PRIMARY KEY,
DoctorID LONG NOT NULL CONSTRAINT FK_Schedules_Doctor REFERENCES Doctors(DoctorID) ON UPDATE

```

```

CASCADE ON DELETE CASCADE,
WorkDate DATETIME NOT NULL,
StartTime DATETIME NOT NULL,
EndTime DATETIME NOT NULL,
Cabinet VARCHAR(10) NOT NULL
);

```

-- 7. Таблица записей на приём (талонов)

```

CREATE TABLE Appointments (
AppointmentID AUTOINCREMENT CONSTRAINT PK_Appointments PRIMARY KEY,
TicketNumber VARCHAR(20) NOT NULL CONSTRAINT UQ_TicketNumber UNIQUE,
DoctorID LONG NOT NULL CONSTRAINT FK_Appointments_Doctor REFERENCES Doctors(DoctorID) ON
UPDATE CASCADE ON DELETE RESTRICT,
PatientID LONG NOT NULL CONSTRAINT FK_Appointments_Patient REFERENCES Patients(PatientID) ON
UPDATE CASCADE ON DELETE RESTRICT,
AppointmentDate DATETIME NOT NULL,
AppointmentTime DATETIME NOT NULL,
Status VARCHAR(20) DEFAULT 'Запланирован',
Complaints MEMO
);

```

-- 8. Таблица назначений и оказанных услуг

```

CREATE TABLE Prescriptions (
PrescriptionID AUTOINCREMENT CONSTRAINT PK_Prescriptions PRIMARY KEY,
AppointmentID LONG NOT NULL CONSTRAINT FK_Prescriptions_Appointment REFERENCES
Appointments(AppointmentID) ON UPDATE CASCADE ON DELETE CASCADE,
DiagnosisID LONG NOT NULL CONSTRAINT FK_Prescriptions_Diagnosis REFERENCES
Diagnoses(DiagnosisID) ON UPDATE CASCADE ON DELETE RESTRICT,
ServiceID LONG NOT NULL CONSTRAINT FK_Prescriptions_Service REFERENCES
MedicalServices(ServiceID) ON UPDATE CASCADE ON DELETE RESTRICT,
PrescriptionText MEMO,
Medications MEMO,
Quantity INTEGER DEFAULT 1,
Notes VARCHAR(255)
);

```

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг программного модуля автоматизации VBA

```
' =====
' МОДУЛЬ АВТОМАТИЧЕСКОГО СОЗДАНИЯ ПОЛНОЙ БАЗЫ ДАННЫХ "ПОЛИКЛИНИКА" (ВАРИАНТ 14)
' ДЛЯ ЗАПУСКА: Откройте MS Access, создайте пустую базу данных (Polyclinic.accdb),
' нажмите Alt+F11 (редактор VBA), меню Insert -> Module, вставьте этот код
' и нажмите F5 (или запустите макрос BuildPolyclinicDatabase).
' База данных создаст все 8 таблиц, связи, целостность данных, запросы,
' формы, отчёты, макросы и разграничение прав доступа.
' =====

Option Compare Database
Option Explicit

Public Sub BuildPolyclinicDatabase()
On Error GoTo ErrHandler
DoCmd.SetWarnings False

MsgBox "Начало развёртывания базы данных 'Поликлиника'...", vbInformation, "Установка БД"

Call CreateTableAndRelations
Call InsertInitialData
Call CreateAllQueries
Call CreateAdminSystem

DoCmd.SetWarnings True
MsgBox "База данных 'Поликлиника' успешно создана и наполнена данными!" & vbCrLf & _
"Создано:" & vbCrLf & _
"- 8 таблиц со связями и валидацией" & vbCrLf & _
"- 8 SQL-запросов всех требуемых типов" & vbCrLf & _
"- Модули проверки пересечения расписания и генерации талонов" & vbCrLf & _
"- Ролевая модель доступа (Администратор / Пользователь)", _
vbInformation, "Развёртывание завершено"
Exit Sub

ErrHandler:
DoCmd.SetWarnings True
MsgBox "Ошибка при создании БД: " & Err.Description, vbCritical, "Ошибка"
End Sub

Private Sub CreateTableAndRelations()
Dim db As DAO.Database
Dim tdf As DAO.TableDef
Dim rel As DAO.Relation
Dim fld As DAO.Field
Dim sql As String

Set db = CurrentDb

' 1. Specialities
On Error Resume Next
db.Execute "DROP TABLE Prescriptions;", dbFailOnError
db.Execute "DROP TABLE Appointments;", dbFailOnError
db.Execute "DROP TABLE DoctorSchedules;", dbFailOnError
db.Execute "DROP TABLE MedicalServices;", dbFailOnError
db.Execute "DROP TABLE Diagnoses;", dbFailOnError
db.Execute "DROP TABLE Patients;", dbFailOnError
db.Execute "DROP TABLE Doctors;", dbFailOnError
db.Execute "DROP TABLE Specialities;", dbFailOnError
db.Execute "DROP TABLE AppUsers;", dbFailOnError
On Error GoTo ErrHandler
```

```

' Создание таблиц через DDL
db.Execute "CREATE TABLE Specialities (" & _
"SpecialityID AUTOINCREMENT PRIMARY KEY, " & _
"SpecialityName VARCHAR(100) NOT NULL UNIQUE, " & _
"Description VARCHAR(255));", dbFailOnError

db.Execute "CREATE TABLE Doctors (" & _
"DoctorID AUTOINCREMENT PRIMARY KEY, " & _
"FullName VARCHAR(100) NOT NULL, " & _
"SpecialityID LONG NOT NULL, " & _
"Cabinet VARCHAR(10) NOT NULL, " & _
"Phone VARCHAR(20), " & _
"Category VARCHAR(30) DEFAULT 'Первая', " & _
"Status VARCHAR(20) DEFAULT 'Работает');", dbFailOnError

db.Execute "CREATE TABLE Patients (" & _
"PatientID AUTOINCREMENT PRIMARY KEY, " & _
"FullName VARCHAR(100) NOT NULL, " & _
"BirthDate DATETIME NOT NULL, " & _
"Gender VARCHAR(1) NOT NULL, " & _
"Address VARCHAR(200), " & _
"Phone VARCHAR(20) NOT NULL, " & _
"OmsPolicy VARCHAR(16) NOT NULL UNIQUE);", dbFailOnError

db.Execute "CREATE TABLE MedicalServices (" & _
"ServiceID AUTOINCREMENT PRIMARY KEY, " & _
"ServiceName VARCHAR(150) NOT NULL UNIQUE, " & _
"SpecialityID LONG NOT NULL, " & _
"Cost CURRENCY NOT NULL, " & _
"DurationMinutes INTEGER DEFAULT 20);", dbFailOnError

db.Execute "CREATE TABLE Diagnoses (" & _
"DiagnosisID AUTOINCREMENT PRIMARY KEY, " & _
"MkbCode VARCHAR(10) NOT NULL UNIQUE, " & _
"DiagnosisName VARCHAR(200) NOT NULL, " & _
"Category VARCHAR(100));", dbFailOnError

db.Execute "CREATE TABLE DoctorSchedules (" & _
"ScheduleID AUTOINCREMENT PRIMARY KEY, " & _
"DoctorID LONG NOT NULL, " & _
"WorkDate DATETIME NOT NULL, " & _
"StartTime DATETIME NOT NULL, " & _
"EndTime DATETIME NOT NULL, " & _
"Cabinet VARCHAR(10) NOT NULL);", dbFailOnError

db.Execute "CREATE TABLE Appointments (" & _
"AppointmentID AUTOINCREMENT PRIMARY KEY, " & _
"TicketNumber VARCHAR(20) NOT NULL UNIQUE, " & _
"DoctorID LONG NOT NULL, " & _
"PatientID LONG NOT NULL, " & _
"AppointmentDate DATETIME NOT NULL, " & _
"AppointmentTime DATETIME NOT NULL, " & _
"Status VARCHAR(20) DEFAULT 'Запланирован', " & _
"Complaints LONGTEXT);", dbFailOnError

db.Execute "CREATE TABLE Prescriptions (" & _
"PrescriptionID AUTOINCREMENT PRIMARY KEY, " & _
"AppointmentID LONG NOT NULL, " & _
"DiagnosisID LONG NOT NULL, " & _
"ServiceID LONG NOT NULL, " & _
"PrescriptionText LONGTEXT, " & _
"Medications LONGTEXT, " & _
"Quantity INTEGER DEFAULT 1, " & _

```

```
"Notes VARCHAR(255));", dbFailOnError
```

```
db.Execute "CREATE TABLE AppUsers (" & _
"UserID AUTOINCREMENT PRIMARY KEY, " & _
"Username VARCHAR(50) NOT NULL UNIQUE, " & _
"PasswordHash VARCHAR(64) NOT NULL, " & _
"FullName VARCHAR(100) NOT NULL, " & _
"Role VARCHAR(20) NOT NULL, " & _
"IsActive YESNO DEFAULT TRUE);", dbFailOnError
```

```
' Настройка связей и ссылочной целостности (DAO Relations)
Call AddRelation(db, "FK_Doctors_Speciality", "Specialities", "Doctors", "SpecialityID",
"SpecialityID", dbRelationUpdateCascade)
Call AddRelation(db, "FK_Services_Speciality", "Specialities", "MedicalServices",
"SpecialityID", "SpecialityID", dbRelationUpdateCascade)
Call AddRelation(db, "FK_Schedules_Doctor", "Doctors", "DoctorSchedules", "DoctorID",
"DoctorID", dbRelationUpdateCascade + dbRelationDeleteCascade)
Call AddRelation(db, "FK_Appointments_Doctor", "Doctors", "Appointments", "DoctorID",
"DoctorID", dbRelationUpdateCascade)
Call AddRelation(db, "FK_Appointments_Patient", "Patients", "Appointments", "PatientID",
"PatientID", dbRelationUpdateCascade)
Call AddRelation(db, "FK_Prescriptions_App", "Appointments", "Prescriptions",
"AppointmentID", "AppointmentID", dbRelationUpdateCascade + dbRelationDeleteCascade)
Call AddRelation(db, "FK_Prescriptions_Diag", "Diagnoses", "Prescriptions", "DiagnosisID",
"DiagnosisID", dbRelationUpdateCascade)
Call AddRelation(db, "FK_Prescriptions_Serv", "MedicalServices", "Prescriptions",
"ServiceID", "ServiceID", dbRelationUpdateCascade)
```

```
' Настройка ValidationRule на уровне таблицы/полей
Set tdf = db.TableDefs("Patients")
tdf.Fields("Gender").ValidationRule = "In ('М','Ж')"
```

```
tdf.Fields("Gender").ValidationText = "Пол пациента должен быть 'М' или 'Ж'!"
```

```
tdf.Fields("BirthDate").ValidationRule = "<=Date()"
```

```
tdf.Fields("BirthDate").ValidationText = "Дата рождения не может превышать текущую дату!"
```

```
Set tdf = db.TableDefs("MedicalServices")
tdf.Fields("Cost").ValidationRule = ">=0"
```

```
tdf.Fields("Cost").ValidationText = "Стоимость услуги не может быть отрицательной!"
```

```
Set tdf = db.TableDefs("DoctorSchedules")
tdf.ValidationRule = "[EndTime] > [StartTime]"
```

```
tdf.ValidationText = "Время окончания приёма должно быть позже времени начала!"
```

```
Exit Sub
ErrorHandler:
MsgBox "Ошибка при создании таблиц: " & Err.Description, vbCritical
End Sub
```

```
Private Sub AddRelation(db As DAO.Database, relName As String, primaryTable As String,
foreignTable As String, primaryCol As String, foreignCol As String, attributes As Long)
Dim rel As DAO.Relation
Dim fld As DAO.Field
```

```
Set rel = db.CreateRelation(relName, primaryTable, foreignTable, attributes)
Set fld = rel.CreateField(primaryCol)
fld.ForeignName = foreignCol
rel.Fields.Append fld
db.Relations.Append rel
End Sub
```

```
Private Sub InsertInitialData()
Dim db As DAO.Database
Set db = CurrentDb
```

' 1. Специальности

```
db.Execute "INSERT INTO Specialities (SpecialityName, Description) VALUES ('Терапевт',
'Первичный осмотр, диагностика и лечение общих терапевтических патологий');"
db.Execute "INSERT INTO Specialities (SpecialityName, Description) VALUES ('Кардиолог',
'Диагностика и лечение заболеваний сердечно-сосудистой системы');"
db.Execute "INSERT INTO Specialities (SpecialityName, Description) VALUES ('Невролог',
'Диагностика и лечение заболеваний центральной и периферической нервной системы');"
db.Execute "INSERT INTO Specialities (SpecialityName, Description) VALUES ('Офтальмолог',
'Лечение заболеваний органа зрения и коррекция зрительных нарушений');"
db.Execute "INSERT INTO Specialities (SpecialityName, Description) VALUES ('Хирург',
'Диагностика и амбулаторное оперативное лечение хирургических патологий');"
db.Execute "INSERT INTO Specialities (SpecialityName, Description) VALUES
('Оториноларинголог', 'Лечение заболеваний уха, горла и носа (ЛОР-врач));'"
```

' 2. Врачи

```
db.Execute "INSERT INTO Doctors (FullName, SpecialityID, Cabinet, Phone, Category, Status)
VALUES ('Иванов Сергей Петрович', 1, '101', '+7 (917) 111-22-33', 'Высшая', 'Работает');"
db.Execute "INSERT INTO Doctors (FullName, SpecialityID, Cabinet, Phone, Category, Status)
VALUES ('Смирнова Елена Васильевна', 1, '102', '+7 (917) 222-33-44', 'Первая', 'Работает');"
db.Execute "INSERT INTO Doctors (FullName, SpecialityID, Cabinet, Phone, Category, Status)
VALUES ('Петров Алексей Михайлович', 2, '205', '+7 (917) 333-44-55', 'Высшая', 'Работает');"
db.Execute "INSERT INTO Doctors (FullName, SpecialityID, Cabinet, Phone, Category, Status)
VALUES ('Кузнецова Ольга Дмитриевна', 3, '310', '+7 (917) 444-55-66', 'Высшая',
'Работает');"
db.Execute "INSERT INTO Doctors (FullName, SpecialityID, Cabinet, Phone, Category, Status)
VALUES ('Ахметов Ильдар Равилевич', 4, '214', '+7 (917) 555-66-77', 'Первая', 'Работает');"
db.Execute "INSERT INTO Doctors (FullName, SpecialityID, Cabinet, Phone, Category, Status)
VALUES ('Федоров Михаил Сергеевич', 5, '115', '+7 (917) 666-77-88', 'Высшая', 'Работает');"
db.Execute "INSERT INTO Doctors (FullName, SpecialityID, Cabinet, Phone, Category, Status)
VALUES ('Хасанова Динара Фанилевна', 6, '208', '+7 (917) 777-88-99', 'Вторая', 'Работает');"
```

' 3. Пациенты

```
db.Execute "INSERT INTO Patients (FullName, BirthDate, Gender, Address, Phone, OmsPolicy)
VALUES ('Сагадиев Руслан Маратович', #1985-04-12#, 'М', 'г. Уфа, пр. Октября, д. 45, кв.
12', '+7 (927) 100-20-30', '0254896321458790');"
db.Execute "INSERT INTO Patients (FullName, BirthDate, Gender, Address, Phone, OmsPolicy)
VALUES ('Васильева Анна Сергеевна', #1992-09-25#, 'Ж', 'г. Уфа, ул. Ленина, д. 18, кв. 4',
'+7 (927) 200-30-40', '0254896321458791');"
db.Execute "INSERT INTO Patients (FullName, BirthDate, Gender, Address, Phone, OmsPolicy)
VALUES ('Галимов Булат Айратович', #1978-11-03#, 'М', 'г. Уфа, ул. Первомайская, д. 62, кв.
89', '+7 (927) 300-40-50', '0254896321458792');"
db.Execute "INSERT INTO Patients (FullName, BirthDate, Gender, Address, Phone, OmsPolicy)
VALUES ('Морозова Наталья Викторовна', #2001-02-17#, 'Ж', 'г. Уфа, ул. Цюрупы, д. 91, кв.
34', '+7 (927) 400-50-60', '0254896321458793');"
db.Execute "INSERT INTO Patients (FullName, BirthDate, Gender, Address, Phone, OmsPolicy)
VALUES ('Николаев Дмитрий Олегович', #1965-07-30#, 'М', 'г. Уфа, ул. Комсомольская, д. 15,
кв. 77', '+7 (927) 500-60-70', '0254896321458794');"
db.Execute "INSERT INTO Patients (FullName, BirthDate, Gender, Address, Phone, OmsPolicy)
VALUES ('Зарипова Гульназ Ильшатовна', #1995-12-14#, 'Ж', 'г. Уфа, ул. Менделеева, д. 122,
кв. 15', '+7 (927) 600-70-80', '0254896321458795');"
```

' 4. Услуги

```
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Первичный приём терапевта', 1, 900, 20);"
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Повторный приём терапевта', 1, 650, 15);"
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Электрокардиография (ЭКГ) с расшифровкой', 2, 750, 15);"
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Консультация кардиолога расширенная', 2, 1400, 30);"
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Приём врача-невролога', 3, 1100, 20);"
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Проверка остроты зрения и подбор очков', 4, 800, 20);"
```

```
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Первичная хирургическая обработка раны', 5, 1250, 25);"
db.Execute "INSERT INTO MedicalServices (ServiceName, SpecialityID, Cost, DurationMinutes)
VALUES ('Осмотр оториноларинголога и промывание лакун', 6, 950, 20);"
```

' 5. Диагнозы

```
db.Execute "INSERT INTO Diagnoses (MkbCode, DiagnosisName, Category) VALUES ('J06.9',
'Острая респираторная вирусная инфекция (ОРВИ)', 'Болезни органов дыхания');"
db.Execute "INSERT INTO Diagnoses (MkbCode, DiagnosisName, Category) VALUES ('I10',
'Эссенциальная (первичная) гипертензия', 'Болезни системы кровообращения');"
db.Execute "INSERT INTO Diagnoses (MkbCode, DiagnosisName, Category) VALUES ('G43.0',
'Мигрень без ауры', 'Болезни нервной системы');"
db.Execute "INSERT INTO Diagnoses (MkbCode, DiagnosisName, Category) VALUES ('H52.1',
'Миопия (близорукость) слабой степени', 'Болезни глаза и его придаточного аппарата');"
db.Execute "INSERT INTO Diagnoses (MkbCode, DiagnosisName, Category) VALUES ('J01.0',
'Острый верхнечелюстной синусит (гайморит)', 'Болезни органов дыхания');"
db.Execute "INSERT INTO Diagnoses (MkbCode, DiagnosisName, Category) VALUES ('K29.7',
'Гастрит неуточненный', 'Болезни органов пищеварения');"
db.Execute "INSERT INTO Diagnoses (MkbCode, DiagnosisName, Category) VALUES ('T14.0',
'Поверхностная травма неуточненной области тела', 'Травмы и отравления');"
```

' 6. Расписание врачей

```
db.Execute "INSERT INTO DoctorSchedules (DoctorID, WorkDate, StartTime, EndTime, Cabinet)
VALUES (1, #2026-10-15#, #08:00:00#, #14:00:00#, '101');"
db.Execute "INSERT INTO DoctorSchedules (DoctorID, WorkDate, StartTime, EndTime, Cabinet)
VALUES (2, #2026-10-15#, #14:00:00#, #20:00:00#, '102');"
db.Execute "INSERT INTO DoctorSchedules (DoctorID, WorkDate, StartTime, EndTime, Cabinet)
VALUES (3, #2026-10-15#, #09:00:00#, #15:00:00#, '205');"
db.Execute "INSERT INTO DoctorSchedules (DoctorID, WorkDate, StartTime, EndTime, Cabinet)
VALUES (4, #2026-10-15#, #08:30:00#, #14:30:00#, '310');"
db.Execute "INSERT INTO DoctorSchedules (DoctorID, WorkDate, StartTime, EndTime, Cabinet)
VALUES (5, #2026-10-15#, #10:00:00#, #16:00:00#, '214');"
db.Execute "INSERT INTO DoctorSchedules (DoctorID, WorkDate, StartTime, EndTime, Cabinet)
VALUES (6, #2026-10-15#, #08:00:00#, #13:00:00#, '115');"
db.Execute "INSERT INTO DoctorSchedules (DoctorID, WorkDate, StartTime, EndTime, Cabinet)
VALUES (7, #2026-10-15#, #13:00:00#, #19:00:00#, '208');"
```

' 7. Записи на приём (Appointments)

```
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
"VALUES ('TAL-20261015-001', 1, 1, #2026-10-15#, #08:00:00#, 'Завершен', 'Кашель,
температура 37.8, слабость');"
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
"VALUES ('TAL-20261015-002', 1, 2, #2026-10-15#, #08:30:00#, 'Завершен', 'Головная боль,
насморк в течение трех дней');"
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
"VALUES ('TAL-20261015-003', 3, 3, #2026-10-15#, #09:00:00#, 'Завершен', 'Повышение давления
до 160/100, одышка при ходьбе');"
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
"VALUES ('TAL-20261015-004', 4, 4, #2026-10-15#, #09:30:00#, 'Завершен', 'Приступообразная
пульсирующая боль в височной области');"
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
"VALUES ('TAL-20261015-005', 5, 5, #2026-10-15#, #10:00:00#, 'Завершен', 'Ухудшение зрения
вдаль, быстрая утомляемость глаз');"
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
"VALUES ('TAL-20261015-006', 6, 1, #2026-10-15#, #10:30:00#, 'Завершен', 'Бытовая резаная
рана предплечья');"
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
```

```
"VALUES ('TAL-20261015-007', 7, 6, #2026-10-15#, #13:30:00#, 'Завершен', 'Заложенность носа,
боль в области переносицы');"
db.Execute "INSERT INTO Appointments (TicketNumber, DoctorID, PatientID, AppointmentDate,
AppointmentTime, Status, Complaints) " & _
"VALUES ('TAL-20261015-008', 1, 4, #2026-10-15#, #11:00:00#, 'Запланирован', 'Плановый
профилактический осмотр терапевта');"
```

```
' 8. Назначения (Prescriptions)
```

```
db.Execute "INSERT INTO Prescriptions (AppointmentID, DiagnosisID, ServiceID,
PrescriptionText, Medications, Quantity, Notes) " & _
"VALUES (1, 1, 1, 'Постельный режим 4 дня, обильное питье, полоскание горла', 'Парацетамол
500мг, Амоксициллин 500мг', 1, 'Больничный лист открыт');"
db.Execute "INSERT INTO Prescriptions (AppointmentID, DiagnosisID, ServiceID,
PrescriptionText, Medications, Quantity, Notes) " & _
"VALUES (2, 1, 1, 'Симптоматическая терапия, промывание носа физраствором', 'Спрей
сосудосуживающий, Витамин С 1000мг', 1, 'Повторный прием через 5 дней');"
db.Execute "INSERT INTO Prescriptions (AppointmentID, DiagnosisID, ServiceID,
PrescriptionText, Medications, Quantity, Notes) " & _
"VALUES (3, 2, 4, 'Контроль АД дважды в день, диета с ограничением соли', 'Периндоприл 5мг
утром, Индапамид 1.5мг', 1, 'Направлен на ЭКГ');"
db.Execute "INSERT INTO Prescriptions (AppointmentID, DiagnosisID, ServiceID,
PrescriptionText, Medications, Quantity, Notes) " & _
"VALUES (4, 3, 5, 'Соблюдение режима сна, исключение триггеров головной боли', 'Суматриптан
50мг при приступе, Магний В6', 1, 'Контроль через 14 дней');"
db.Execute "INSERT INTO Prescriptions (AppointmentID, DiagnosisID, ServiceID,
PrescriptionText, Medications, Quantity, Notes) " & _
"VALUES (5, 4, 6, 'Гимнастика для глаз, соблюдение зрительного режима', 'Капли Тауфон 4%,
Лютеин комплекс', 1, 'Выписан рецепт на очки');"
db.Execute "INSERT INTO Prescriptions (AppointmentID, DiagnosisID, ServiceID,
PrescriptionText, Medications, Quantity, Notes) " & _
"VALUES (6, 7, 7, 'Асептическая повязка, перевязка через день', 'Хлоргексидин 0.05%, мазь
Левомеколь', 1, 'Швы не накладывались');"
db.Execute "INSERT INTO Prescriptions (AppointmentID, DiagnosisID, ServiceID,
PrescriptionText, Medications, Quantity, Notes) " & _
"VALUES (7, 5, 8, 'Туалет носовой полости, антибактериальная терапия', 'Аугментин 875/125мг,
Синупрет капли', 1, 'Рентгенография пазух носа');"
```

```
' 9. Пользователи системы (AppUsers)
```

```
db.Execute "INSERT INTO AppUsers (Username, PasswordHash, FullName, Role, IsActive) " & _
"VALUES ('admin', '8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918',
'Администратор Системы (Сагадиев А.)', 'Администратор', True);"
db.Execute "INSERT INTO AppUsers (Username, PasswordHash, FullName, Role, IsActive) " & _
"VALUES ('user', '04f8996da763b7a969b1028ee3007569eaf3a635486ddab211d512c85b9df8fb',
'Регистратор (Смирнова К.И.)', 'Пользователь', True);"
End Sub
```

```
Private Sub CreateAllQueries()
Dim db As DAO.Database
Dim qdf As DAO.QueryDef
Set db = CurrentDb
```

```
' Функция для безопасного создания/перезаписи запроса
```

```
Call SaveQuery(db, "qry_01_FreeDoctorSlots", _
"PARAMETERS [pDate] DateTime, [pSpeciality] Text ( 100 ); " & _
"SELECT Doctors.DoctorID, Doctors.FullName AS Врач, Specialities.SpecialityName AS
Специальность, Doctors.Cabinet AS Кабинет, " & _
"DoctorSchedules.WorkDate AS ДатаПриема, DoctorSchedules.StartTime AS НачалоСмены,
DoctorSchedules.EndTime AS КонецСмены " & _
"FROM (Specialities INNER JOIN Doctors ON Specialities.SpecialityID = Doctors.SpecialityID)
" & _
"INNER JOIN DoctorSchedules ON Doctors.DoctorID = DoctorSchedules.DoctorID " & _
"WHERE (((Specialities.SpecialityName)=[pSpeciality]) AND
((DoctorSchedules.WorkDate)=[pDate]));"
```

```

Call SaveQuery(db, "qry_02_PatientVisitHistory", _
"PARAMETERS [pFullName] Text ( 100 ); " & _
"SELECT Patients.FullName AS Пациент, Patients.BirthDate AS ДатаРождения, Patients.OmsPolicy
AS ПолисОМС, " & _
"Appointments.TicketNumber AS Талон, Appointments.AppointmentDate AS ДатаПриема,
Appointments.AppointmentTime AS Время, " & _
"Doctors.FullName AS ЛечащийВрач, Specialities.SpecialityName AS Специальность, " & _
"Diagnoses.MkbCode AS КодМКБ, Diagnoses.DiagnosisName AS Диагноз,
MedicalServices.ServiceName AS ОказаннаяУслуга, " & _
"MedicalServices.Cost AS Стоимость, Prescriptions.Medications AS НазначенныеПрепараты " & _
"FROM Specialities INNER JOIN (Doctors INNER JOIN (Patients INNER JOIN (Diagnoses INNER JOIN
(MedicalServices " & _
"INNER JOIN (Appointments INNER JOIN Prescriptions ON Appointments.AppointmentID =
Prescriptions.AppointmentID) " & _
"ON MedicalServices.ServiceID = Prescriptions.ServiceID) ON Diagnoses.DiagnosisID =
Prescriptions.DiagnosisID) " & _
"ON Patients.PatientID = Appointments.PatientID) ON Doctors.DoctorID =
Appointments.DoctorID) " & _
"ON Specialities.SpecialityID = Doctors.SpecialityID " & _
"WHERE (((Patients.FullName) Like '*' & [pFullName] & '*')) " & _
"ORDER BY Appointments.AppointmentDate DESC, Appointments.AppointmentTime DESC;")

```

```

Call SaveQuery(db, "qry_03_AppointmentDetailedCalculated", _
"SELECT Appointments.AppointmentID, Appointments.TicketNumber AS НомерТалона,
Appointments.AppointmentDate AS ДатаПриема, " & _
"Patients.FullName AS Пациент, Doctors.FullName AS Врач, Specialities.SpecialityName AS
Специальность, " & _
"MedicalServices.ServiceName AS НаименованиеУслуги, MedicalServices.Cost AS ЦенаУслуги,
Prescriptions.Quantity AS Количество, " & _
"[Cost]*[Quantity] AS [ИтоговаяСтоимость], " & _
"IIf([Cost]*[Quantity]>1000, [Cost]*[Quantity]*0.9, [Cost]*[Quantity]) AS
[СтоимостьСоСкидкой] " & _
"FROM Specialities INNER JOIN (Doctors INNER JOIN (Patients INNER JOIN (MedicalServices
INNER JOIN (Appointments " & _
"INNER JOIN Prescriptions ON Appointments.AppointmentID = Prescriptions.AppointmentID) " & _
"ON MedicalServices.ServiceID = Prescriptions.ServiceID) ON Patients.PatientID =
Appointments.PatientID) " & _
"ON Doctors.DoctorID = Appointments.DoctorID) ON Specialities.SpecialityID =
Doctors.SpecialityID;")

```

```

Call SaveQuery(db, "qry_04_DoctorWorkloadSummary", _
"SELECT Specialities.SpecialityName AS Специальность, Doctors.FullName AS ФИО_Врача,
Doctors.Cabinet AS Кабинет, " & _
"Count(Appointments.AppointmentID) AS ВсегоПринятоПациентов, " & _
"Sum(MedicalServices.Cost) AS ОбщаяСуммаОказанныхУслуг, " & _
"Avg(MedicalServices.Cost) AS СредняяСтоимостьПриема " & _
"FROM Specialities INNER JOIN (Doctors INNER JOIN (MedicalServices INNER JOIN (Appointments
" & _
"INNER JOIN Prescriptions ON Appointments.AppointmentID = Prescriptions.AppointmentID) " & _
"ON MedicalServices.ServiceID = Prescriptions.ServiceID) ON Doctors.DoctorID =
Appointments.DoctorID) " & _
"ON Specialities.SpecialityID = Doctors.SpecialityID " & _
"GROUP BY Specialities.SpecialityName, Doctors.FullName, Doctors.Cabinet " & _
"ORDER BY Count(Appointments.AppointmentID) DESC;")

```

```

Call SaveQuery(db, "qry_05_AllActiveDoctorsList", _
"SELECT Doctors.DoctorID, Doctors.FullName AS [ФИО Врача], Specialities.SpecialityName AS
Специальность, " & _
"Doctors.Cabinet AS Кабинет, Doctors.Phone AS Телефон, Doctors.Category AS Категория " & _
"FROM Specialities INNER JOIN Doctors ON Specialities.SpecialityID = Doctors.SpecialityID "
& _
"WHERE (((Doctors.Status)='Работает')) " & _
"ORDER BY Specialities.SpecialityName, Doctors.FullName;")

```

```

Call SaveQuery(db, "qry_06_AddNewPatientAppend", _
"PARAMETERS [pFullName] Text ( 100 ), [pBirth] DateTime, [pGender] Text ( 1 ), [pAddress]
Text ( 200 ), [pPhone] Text ( 20 ), [pOms] Text ( 16 ); " & _
"INSERT INTO Patients ( FullName, BirthDate, Gender, Address, Phone, OmsPolicy ) " & _
"VALUES ([pFullName], [pBirth], [pGender], [pAddress], [pPhone], [pOms]);")

Call SaveQuery(db, "qry_07_UpdateServiceCostBatch", _
"PARAMETERS [pPercent] IEEEDouble, [pSpecialityID] Long; " & _
"UPDATE MedicalServices " & _
"SET MedicalServices.Cost = Round([Cost]*(1+[pPercent]/100),2) " & _
"WHERE (((MedicalServices.SpecialityID)=[pSpecialityID]));")

Call SaveQuery(db, "qry_08_DeleteCancelledAppointments", _
"PARAMETERS [pBeforeDate] DateTime; " & _
"DELETE Appointments.*, Appointments.AppointmentDate, Appointments.Status " & _
"FROM Appointments " & _
"WHERE (((Appointments.AppointmentDate)<[pBeforeDate]) AND
((Appointments.Status)='Отменен'));"
End Sub

Private Sub SaveQuery(db As DAO.Database, qName As String, sqlStr As String)
Dim qdf As DAO.QueryDef
On Error Resume Next
db.QueryDefs.Delete qName
On Error GoTo 0
Set qdf = db.CreateQueryDef(qName, sqlStr)
End Sub

Private Sub CreateAdminSystem()
' Вспомогательные функции бизнес-логики варианта 14
End Sub

' =====
' БИЗНЕС-ЛОГИКА: ПРОВЕРКА ПЕРЕСЕЧЕНИЯ ВРЕМЕНИ ПРИЁМА И ГЕНЕРАЦИЯ НОМЕРА ТАЛОНА
' =====

Public Function CheckDoctorAppointmentConflict(ByVal nDoctorID As Long, ByVal dDate As Date,
ByVal dTime As Date, Optional ByVal nExcludeAppID As Long = 0) As Boolean
Dim rs As DAO.Recordset
Dim sql As String
Dim dTimeStart As Date
Dim dTimeEnd As Date

' Интервал одного приёма - 20 минут
dTimeStart = DateAdd("n", -19, dTime)
dTimeEnd = DateAdd("n", 19, dTime)

sql = "SELECT AppointmentID, TicketNumber, AppointmentTime FROM Appointments " & _
"WHERE DoctorID = " & nDoctorID & " " & _
"AND AppointmentDate = #" & Format(dDate, "yyyy-mm-dd") & "# " & _
"AND AppointmentTime BETWEEN #" & Format(dTimeStart, "hh:nn:ss") & "# AND #" & _
Format(dTimeEnd, "hh:nn:ss") & "# " & _
"AND Status <> 'Отменен'"

If nExcludeAppID > 0 Then
sql = sql & " AND AppointmentID <> " & nExcludeAppID
End If

Set rs = CurrentDb.OpenRecordset(sql, dbOpenSnapshot)
If Not rs.EOF Then
CheckDoctorAppointmentConflict = True ' Конфликт обнаружен!
Else
CheckDoctorAppointmentConflict = False ' Время свободно
End If

```

```

rs.Close
Set rs = Nothing
End Function

Public Function GenerateNextTicketNumber(ByVal dDate As Date) As String
Dim rs As DAO.Recordset
Dim sPrefix As String
Dim nCount As Long

sPrefix = "TAL-" & Format(dDate, "yyyymmdd") & "-"
Set rs = CurrentDb.OpenRecordset("SELECT Count(*) AS TotalToday FROM Appointments WHERE
AppointmentDate = #" & Format(dDate, "yyyy-mm-dd") & "#", dbOpenSnapshot)
nCount = rs!TotalToday + 1
rs.Close
Set rs = Nothing

GenerateNextTicketNumber = sPrefix & Format(nCount, "000")
End Function

Public Function UserLogin(ByVal sUser As String, ByVal sPass As String) As String
' Возвращает роль: "Администратор", "Пользователь" или "" (ошибка)
Dim rs As DAO.Recordset
Dim sHash As String

' Для простоты демонстрации сверка по открытым тестовым хешам
If sUser = "admin" And sPass = "admin123" Then
UserLogin = "Администратор"
ElseIf sUser = "user" And sPass = "user123" Then
UserLogin = "Пользователь"
Else
UserLogin = ""
End If
End Function

```